

TaskVM: Agentic GenUI × GUI Agent

相关工作撞车地图

面向 CHI 论文写作与 W2/W3 路线决策的可编译材料包

2026 年 8 月 6 日

一句话结论

“首次结合 Agentic GenUI 与 GUI Agent” 已经不能成立。Morae、AgentLens 与 ALLOY 分别占据了“决策点生成界面”“GUI Agent 的即时 GenUI 沟通模式”“可编辑任务流程由浏览器 Agent 执行”。TaskVM 应守住更窄但更强的贡献：**把多个既有应用的实时状态编译为持久、可编辑、可执行的任务投影；用户编辑被写回真实应用，并由独立 verifier 检查目标效果、非干涉与重新同步。**

目录

1	范围、基准定义与最重要的修正	3
1.1	本报告讨论的“Agentic GenUI”	3
1.2	TaskVM 的不可替换四锚点	3
1.3	对此前说法的正式修正	3
2	分级方法	3
2.1	六个判据	3
2.2	等级定义	4
3	S 级撞车：已经把 Agentic GenUI 与 GUI Agent 放进同一闭环	4
3.1	S1 Morae：从当前真实 UI 状态生成决策界面	4
3.2	S2 ALLOY：持久、可编辑的任务级工作流由 Web Agents 执行	5
3.3	S3 AgentLens：明确提出 GenUI 作为 Mobile GUI Agent 的交互模式	5
3.4	S 级总判断	5
4	A 级撞车：只缺一到两个关键连接	6
4.1	A1 AOHP：跨应用服务组合与 Generated Service Entrances	6
4.2	A2 A-Mash：多个未修改 App 的同步、可操作单界面投影	6
4.3	A3 Jelly：task-driven data model 驱动持久、可编辑 GenUI	6
4.4	A4 DuetUI：用户操作生成 UI，反向塑造 Agent	6
4.5	A5 Software as Content：持久动态应用成为交互状态	7
4.6	A6 Macaron-A2UI：Assistant-side Generative UI 的模型与评测体系	7
4.7	A7 TaskLens：为既有专业软件生成 task-conditioned scaffolded UI	7

4.8	A8 MAESTRO: 基于偏好的 GUI 适配与 workflow 回退	7
5	B 级撞车: 占据一个核心子空间	7
5.1	B1 Spatula: 按需生成 in-situ 参数控制 UI	7
5.2	B2 AUI-Gym / Computer-Use Agents as Judges for Generative UI	8
5.3	B3 WebVIA / WebGen-Agent 类: GUI Agent 探索或测试后生成网站	8
5.4	B4 Generative Interfaces for Language Models / Portal UX Agent	8
5.5	B5 A2UI、MCP Apps、AG-UI、CopilotKit、LangGraph Generative UI	8
5.6	B6 SUGILITE / PUMICE / End-user Programming by Demonstration	8
5.7	B7 ReUseIt 与可复用 Web Agent 工作流	8
5.8	B8 Generated personal software / app-less engineering demos	8
6	C 级与 D 级: 相邻谱系与技术底座	9
6.1	C 级: GUI Agent + 人类协作, 但没有 Agentic GenUI	9
6.2	D 级: benchmark、验证与基础设施	9
7	核心比较矩阵	10
8	论文可用 Claim 与禁用 Claim	11
8.1	禁用	11
8.2	推荐主 Claim	11
8.3	一句话防守卡	12
8.4	Teaser / Demo 必须演出的四步	12
9	A2UI 协议版本: 与撞车地图相互独立的工程决策	12
9.1	事实更新	12
9.2	对 TaskVM 的推荐	12
10	落地行动清单	13
10.1	Related Work 主文结构	13
10.2	W2 开工指令应增加的三项	13
10.3	下一轮文献审计优先级	13
11	观察名单: 保留但暂不作为撞车证据	13

1 范围、基准定义与最重要的修正

1.1 本报告讨论的“Agentic GenUI”

这里不使用宽泛的“agentic UI”，也不把给 Agent 使用的 dashboard、轨迹面板或 agent-friendly GUI 算作 GenUI。纳入核心撞车判定的系统需要尽量满足：

1. Agent 在运行时生成或重构给人操作的界面；
2. 用户在该界面上的操作进入 Agent 的后续执行；
3. Agent 通过 GUI/Computer-Use 方式操作既有软件，或至少执行真实浏览器/移动应用工作流；
4. 生成界面不是纯 mock-up，而与任务执行上下文或应用状态有实质联系。

1.2 TaskVM 的不可替代四锚点

TASKVM 的锁定主线是：

existing applications + live state + executable binding + round-trip verification.

其八步闭环为：

Observe → Abstract → Project → Manipulate → Compile → Execute → Verify → Reconcile.

真正要证明的不是“模型会生成控件”，而是：

多个既有应用中的真实状态能否被编译为一个任务级投影；用户修改投影后，目标变化是否在真实应用中发生、无关状态是否保持不动、投影是否随后与 ground truth 重新同步。

1.3 对此前说法的正式修正

以下表述应从论文、开工指令和口头介绍中删除：

- “No prior work combines Generative UI with GUI agents.”
- “首次让人通过 Agent 生成的界面控制 GUI Agent。”
- “首次生成统一任务入口并跨应用执行。”
- “首次用持久动态应用作为人-Agent 交互层。”

可守的版本必须同时带上：**live cross-application state、state projection、write-back、independent verification、reconciliation。**

2 分级方法

2.1 六个判据

符号	判据	判定问题
----	----	------

G	Runtime GenUI	是否由 Agent 在运行时生成/重构面向人的可交互 UI?
U	User operation	用户能否直接操作该生成 UI，且操作改变后续系统行为?
E	Existing-software execution	是否有 GUI/Computer-Use Agent 操作真实或未修改的既有软件?
L	Live-state projection	生成 UI 是否持续代表底层真实应用状态，而非只表达询问、计划或内部模型?
X	Cross-application binding	同一任务变量是否绑定并传播到多个应用/服务?
V	Independent round trip	是否由独立 ground truth 验证目标效果、非干涉，并执行 reconciliation?

2.2 等级定义

- **S 级**: 严格组合已成立。至少 G+U+E 形成真实闭环；必须在主文正面引用并逐条切割。
- **A 级**: 只缺一个或两个承重连接，或在整体系统形态上极接近。审稿人很可能视为直接近邻。
- **B 级**: 占据 GenUI、持久任务 UI、跨应用聚合、可执行 workflow 或 UI 生成协议中的一个核心子空间。
- **C 级**: GUI Agent 的人机协作、暂停、监督、自然语言控制与跨应用执行相关，但没有 Agentic GenUI。
- **D 级**: 执行后端、benchmark、验证器、协议和工程基础设施；不是交互 novelty 竞品，但会影响技术定位。

证据纪律

本报告把“撞车等级”与“发表状态”分开。同行评审论文、arXiv 预印本、工程框架和仅有公开演示的项目可以都很相关，但不能以同样强度写进 novelty claim。未完成原文核验的条目被移入观察名单，不作为“已有工作已经完成某机制”的证据。

3 S 级撞车：已经把 Agentic GenUI 与 GUI Agent 放进同一闭环

3.1 S1 Morae：从当前真实 UI 状态生成决策界面

工作。Morae 在 Web UI Agent 执行过程中分析用户请求、当前 UI 表示、截图和动作历史，识别需要用户偏好或澄清的决策点，暂停自动化并动态生成可访问的交互控件；用户选择后，Agent 继续操作真实网站。[2]

撞车点。

- 生成界面来自真实网页执行上下文，而不是独立聊天；
- 用户直接操作生成控件；
- 该操作决定 GUI Agent 后续在真实网页中的动作。

缺口。其 UI 是一次性的 decision-point surface，不是多应用实时状态的持久投影；没有跨应用 task-variable binding、non-interference、独立 ground-truth round trip 或 reconciliation。

最锋利切割句。

Morae projects a choice; TASKVM projects the live state of a task distributed across applications.

不能再 claim: “首次从当前 GUI 上下文生成可操作界面，并让选择驱动 GUI Agent。”

3.2 S2 ALLOY: 持久、可编辑的任务级工作流由 Web Agents 执行

工作。 ALLOY 从用户在浏览器中的演示自动推断图结构的任务级工作流。工作流持续可见、可编辑、可保存和复用；每个节点背后由浏览器 Agent 执行。[4]

撞车点。

- Agent 自动生成高层、持久、可编辑的任务表示；
- 用户无需编辑点击轨迹，可直接增删节点和依赖；
- 编辑后的表示由真实浏览器 Agents 执行。

缺口。 ALLOY 投影的是 **procedure/program**，不是多个应用的当前世界状态；它不要求 task variable 与外部对象持续双向同步，也没有 expected state diff、非干涉和 reconciliation。

最锋利切割句。

ALLOY lets users edit the program executed by browser agents; TASKVM lets users edit the task state that existing applications must realize.

不能再 claim: “首次让用户编辑 Agent 生成的任务级界面，并由 Web Agents 执行。”

3.3 S3 AgentLens: 明确提出 GenUI 作为 Mobile GUI Agent 的交互模态

工作。 AgentLens 让 Mobile GUI Agent 在 Virtual Display 中操作未修改的第三方应用，并在需要人交互时选择 Full UI、Partial UI 或 GenUI。GenUI 由独立 LLM 生成 HTML，用户回应后主 GUI Agent 继续执行。[3]

撞车点。

- 论文明确把 GenUI 与 Mobile GUI Agent 放在同一系统中；
- 生成界面承担询问、确认、摘要和执行交接；
- 用户操作进入真实 App 的后续自动化。

缺口。 其 GenUI 是即时 communication/handoff overlay，且在真实性或高风险步骤会退回 Partial/Full UI；不是持久 task-state surface，也无跨应用语义绑定与独立 verifier。

最锋利切割句。

AgentLens generates a communication surface between a user and a GUI agent; TASKVM generates a control surface between a user and the underlying software state.

3.4 S 级总判断

三个 S 级近邻已经占据了三个不同位置：

工作	主要投影对象	生成 UI 的角色
Morae	choice / ambiguity	决策点选择器
AgentLens	conversation / handoff	即时视觉沟通模态
ALLOY	procedure / workflow	可编辑 Agent 程序
TASKVM	live software-world state	持久、可执行、可验证状态控制面

4 A 级撞车：只缺一到两个关键连接

4.1 A1 AOHP：跨应用服务组合与 Generated Service Entrances

AOHP 在 AOSP 层把 Agent 设为 OS 一等角色，支持 personalized service composition、generated service entrances、结构化 UI、Rendered GUI、API/CLI 等多种执行接口。其“购物入口”等任务级界面可聚合多个服务并暴露比较和购买操作。[5]

为什么危险：已经具备“任务入口 + 多真实应用/服务 + Agent 执行”的整体形态。

为何仍非 TaskVM：它优化的是 Agent-native OS harness 与服务组合，没有从多个应用 ground truth 反向编译持续任务状态，也没有用户编辑 task variable 后的 expected diff、non-interference 和 reconciliation verifier。

切割：AOHP composes services; TASKVM maintains a verified projection of distributed application state.

4.2 A2 A-Mash：多个未修改 App 的同步、可操作单界面投影

A-Mash 从未修改 Android App 动态提取 UI，将不同 App 的控件嵌入统一 wrapper，并保持同步，形成 single-app illusion。[6]

为什么危险：它已经占据“多个真实既有 App → 一个同步且可操作界面”。

缺口：没有 LLM/Agent 自动任务抽象、task semantic variable、依赖传播、GUI Agent 代执行或独立 verifier。

切割：A-Mash remixes application widgets; TASKVM compiles task semantics and verifies their effects.

4.3 A3 Jelly：task-driven data model 驱动持久、可编辑 GenUI

Jelly 由 prompt 生成 object-relational schema 和 dependency graph，再映射为 UI；用户通过自然语言或直接操作修改 UI，修改被翻译为底层 task-driven model 的变化。[7]

为什么危险：其“task model → persistent UI → direct manipulation → model update”与 TASKVM 的中间表示和界面机制高度同构。

缺口：Jelly 的内部模型是主要状态；TASKVM 的 task state 只是外部应用状态的绑定投影，source of truth 永远留在真实应用中。

4.4 A4 DuetUI：用户操作生成 UI，反向塑造 Agent

DuetUI 通过 bidirectional context loop 让 Agent 逐步搭建任务 UI；用户直接操作界面，操作继续影响 Agent 的任务分解和下一轮生成。[8]

为什么危险：已经占据 “direct manipulation of generated UI steers agent behavior”。

缺口：其核心是 top-down intent co-generation；外部服务执行在原始实现中为模拟/抽象，没有 bottom-up live application state、真实跨应用写回和独立 verifier。

4.5 A5 Software as Content：持久动态应用成为交互状态

SaC 把动态生成、持续演化的 agentic application 设为人-Agent 的主要交互层。[9]

为什么危险：它已占据 “persistent dynamic software as the human-agent interaction layer” 这一父范式。

缺口：其 application 本身构成交互状态；公开实现承认 write-capable execution、外部系统事务一致性和 frontend-ground-truth synchronisation 尚未解决。TASKVM 的核心恰是 surface 永不成为真理来源。

4.6 A6 Macaron-A2UI：Assistant-side Generative UI 的模型与评测体系

Macaron-A2UI 训练模型生成轻量、可执行的 UI actions，用于信息收集、偏好细化、确认和多目标组织，并提供 A2UI-Bench。[10]

为什么危险：它会占据 “Agent 生成声明式可执行控件” 的模型与 benchmark 叙事，且名称容易与项目历史代号混淆。

缺口：其研究目标明确不覆盖对既有界面的动作执行；TASKVM 不应把 widget generation 当贡献，而应把 A2UI 视作可替换的 wire format。

4.7 A7 TaskLens：为既有专业软件生成 task-conditioned scaffolded UI

TaskLens 从自然语言任务描述识别 workflow stages、domain concepts 和相关工具，生成并执行实现现代码，为 Blender 等专业软件产生任务条件化的 scaffolded interface。[11]

为什么危险：它直接从任务生成覆盖既有复杂软件的定制界面，并在真实软件学习任务中验证。

缺口：其目标是教学与工具渐进显露，不是 GUI Agent 在多个应用中维护 live state、把用户编辑写回底层状态并做 round-trip verification。

4.8 A8 MAESTRO：基于偏好的 GUI 适配与 workflow 回退

MAESTRO 在带 GUI 的对话式 Agent 中维护偏好记忆，执行 augment/sort/filter/highlight 等界面适配，并在偏好冲突时建议 workflow 回退。[12]

为什么危险：它把 Agent 从 “翻译输入为 GUI 动作” 扩展到运行时 UI adaptation 与多阶段决策支持。

缺口：它主要在受控 booking GUI 内调整既有界面，不是通用 GUI Agent over unmodified apps，也不生成跨应用 live-state projection。

5 B 级撞车：占据一个核心子空间

5.1 B1 Spatula：按需生成 in-situ 参数控制 UI

Spatula 为 motion graphics 的属性控制按需生成局部、原位的控制界面，并探索控制粒度、范围与可扩展性。[13] 它证明 “运行时生成可直接操作的参数面板” 本身已不是空白，但没有跨应用 GUI Agent、外部 ground truth 或 write-back verifier。

5.2 B2 AUI-Gym / Computer-Use Agents as Judges for Generative UI

这类系统让 Coder Agent 生成网站，再让 CUA 在生成网站中执行任务和评估可操作性。它们组合了 GenUI 与 CUA，但拓扑方向相反：CUA 操作**生成的网站**用于设计/评测，而不是人操作生成 UI 来控制既有软件。

5.3 B3 WebVIA / WebGen-Agent 类：GUI Agent 探索或测试后生成网站

这些工作使用 Agent 探索目标网站、捕获状态、生成交互实现，并通过截图或 GUI 测试反馈迭代。它们证明 GUI exploration 可以给 UI generation 提供输入，也可自动验证交互，但生成界面通常是复刻/开发产物，不与被观察网站持续双向绑定。

5.4 B4 Generative Interfaces for Language Models / Portal UX Agent

此类工作研究 LLM 如何根据 query 或结构化任务生成专用 UI，常用可信组件集合、typed templates 或确定性 renderer。它们覆盖 bounded GenUI，但没有真实既有应用的观察、GUI write-back 和 round-trip correctness。

5.5 B5 A2UI、MCP Apps、AG-UI、CopilotKit、LangGraph Generative UI

这些协议和框架让 Agent 返回声明式组件或交互 HTML，并把用户事件回传 Agent runtime。它们是实现 substrate，不是 TASKVM 的研究反例：

- 它们定义 Agent-frontend 的消息和状态同步；
- 不定义如何观察第三方 App 的 ground truth；
- 不定义 task-variable 到异质应用对象的 binding；
- 不提供 non-interference 或 reconciliation 的独立判定。

5.6 B6 SUGILITE / PUMICE / End-user Programming by Demonstration

这条谱系长期研究如何从自然语言、示范和第三方 GUI 中学习可复用自动化及用户概念。其意义是：“高层语义表示 → 真实 GUI 执行”本身不能作为 novelty。TASKVM 的新意必须落在由 **live state** 编译出的可编辑投影及其 **verified round trip**。

5.7 B7 ReUseIt 与可复用 Web Agent workflow

ReUseIt 从 Agent 的成功/失败尝试合成带 execution guards 的可复用 Web workflow，提高重复任务可靠性并让用户理解失败。[\[14\]](#) 它与 ALLOY 一样主要投影程序/执行策略，而非跨应用世界状态。

5.8 B8 Generated personal software / app-less engineering demos

Moldable、OpenUI/AppLess、OpenGenerativeUI 等工程项目展示了实时生成持久应用或界面的趋势。除非公开证据明确证明其调用通用 GUI Agent 操作未修改第三方软件并保持状态绑定，否则只能作为产业趋势或实现示例，不能当作严格学术撞车证据。

6 C 级与D 级：相邻谱系与技术底座

6.1 C 级：GUI Agent + 人类协作，但没有 Agentic GenUI

工作/谱系	已占据的空间	与 TASKVM 的差异
Sidekick	CUA 执行的外围反馈、摘要、错误提示与暂停	压缩 actions，不投影 applications；不执行、不写回
Magentic-UI	计划审阅、action guards、暂停、接管、workspace	Agent dashboard/oversight，不是 task-state surface
CowPilot / MIWA	执行中的 mixed-initiative steering 与人工接管	主要操作原 UI 或轨迹，不生成新的任务状态界面
WebLINX / META-GUI	多轮人-Agent 对话驱动真实 Web/Mobile GUI	交互主要是语言与原界面动作
Savant / JustSpeak 等	统一自然语言入口控制异质 GUI	统一 command layer，不是 live visual projection
UFO / UFO2	Windows 跨应用 Agent、GUI/API hybrid execution	可作为 actuator/backend，无面向人的 GenUI 投影
EmbeWebAgent	Agent 与 live Web UI 的嵌入和高层动作注册	把 Agent 放入现有 UI，而非生成新的任务界面
ActionEngine	GUI 状态机记忆与完整可执行程序生成	中间表示供 Agent 使用，不给人直接操作
OS-Kairos / VeriOS 类	适时人工介入、错误检测、恢复和验证	验证 trajectory/outcome，不验证 projection fidelity
ScreenAgent / ShowUI 等	截图到鼠标键盘动作的通用 GUI 执行	仅执行模型，不构成 HCI 交互竞品

6.2 D 级：benchmark、验证与基础设施

工作/谱系	可借鉴点	不构成撞车的原因
MyPCBench / WindowsWorld	多应用、登录态、个人数据、跨应用任务	评测 GUI Agents，不生成用户任务 UI
OmniACT / Browser-Gym 类	可执行桌面/Web 动作与标准化环境	actuator/benchmark 层
OpenApps	rename/reskin/layout/content 变化下的 OOD 鲁棒性	可支持 TASKVM 的反捷径 split
UI-Genie / outcome verification	结果判定、hard negative、自提升	verifier 对象是 Agent outcome，不是 UI round trip

InfiniteWeb	自动生成网站环境与 programmatic evaluator	训练/评测环境, 不是现有应用的用户投影
A2UI v0.9.1	安全、声明式、prompt-first 的 UI wire format	不定义任务状态、执行和 ground truth
MCP Apps / AG-UI	Agent 与 frontend/tool 的事件和状态通道	transport, 不是 cross-app binding
Playwright / Virtual Display	Web/Mobile 执行与后台界面承载	工程实现底座

7 核心比较矩阵

工作	G	U	E	L	X	V	最关键差异
Morae	✓	✓	✓	△	✗	✗	临时决策点 UI
AgentLens	✓	✓	✓	△	✗	✗	即时沟通/交接模式
ALLOY	✓	✓	✓	✗	△	✗	投影 procedure, 不投影 world state
AOHP	✓	✓	✓	△	✓	✗	OS 服务组合, 无 round-trip fidelity
A-Mash	✗	✓	✗	✓	✓	✗	原控件 mashup, 无 Agent 语义编译
Jelly	✓	✓	✗	△	✗	✗	内部 data model 是 source of truth
DuetUI	✓	✓	✗	✗	△	✗	intent co-generation, 服务抽象/模拟
SaC	✓	✓	✗	△	✗	✗	generated app 本身就是 state
Macaron-A2UI	✓	✓	✗	✗	✗	✗	只覆盖 assistant-side GenUI
TaskLens	✓	✓	✗	△	✗	✗	教学 scaffold, 不是 Agent write-back
MAESTRO	△	✓	△	△	✗	✗	受控 GUI 内偏好适配
Spatula	✓	✓	✗	✗	✗	✗	生成内容参数控制 UI
TaskVM	✓	✓	✓	✓	✓	✓	多应用 live state 的 verified executable projection

说明: ✓= 明确具备; △= 部分/受限/语义不同; ✗= 未发现。矩阵是研究定位工具, 不是对各系统质量的价值判断。

8 论文可用 Claim 与禁用 Claim

8.1 禁用

不要写

- We are the first to combine Generative UI with GUI agents.
- We are the first to let users manipulate generated interfaces to steer UI agents.
- We are the first to generate task-level entrances over multiple applications.
- We are the first to use persistent dynamic applications as the human-agent interaction layer.
- We are the first to unify multiple existing applications in one operable interface.

8.2 推荐主 Claim

英文主版本

Prior work has generated decision interfaces for UI agents, used GenUI as a just-in-time communication modality for mobile GUI agents, and created editable workflows executed by web agents. TASKVM instead generates a persistent executable projection of live state distributed across existing applications. Users edit task-state variables rather than agent procedures; the resulting cross-application effects, non-interference, and reconciliation are independently verified against application ground truth.

中文主版本

既有工作已经为 UI Agent 生成即时决策界面，将 GenUI 用作移动 GUI Agent 的视觉沟通模态，并生成可编辑、可由 Web Agents 执行的任务工作流。TASKVM 生成的则是分布于多个既有应用中的实时任务状态之持久可执行投影：用户修改的是软件世界应达到的任务状态，而不是 Agent 的执行程序；系统随后依据应用 ground truth 独立验证跨应用效果、非干涉与重新同步。

8.3 一句话防守卡

对手	一句话区分
Morae	Projects a choice; TASKVM projects live distributed task state.
AgentLens	Communication surface vs. software-state control surface.
ALLOY	Procedure projection vs. state projection.
AOHP	Service composition vs. verified state projection.
A-Mash	Widget mashup vs. semantic compilation and verified effects.
Jelly	Internal model as state vs. external applications as source of truth.
DuetUI	Top-down intent co-generation vs. bottom-up live-state compilation.
SaC	The app is state vs. the surface is never the source of truth.
Sidekick	Compresses actions vs. compresses applications.
Macaron-A2UI	Generates UI actions vs. binds UI edits to existing software effects.

8.4 Teaser / Demo 必须演出的四步

1. 用户在任务面把发布日期从 8/14 支为 8/18;
2. 真实 Calendar、TaskBoard、Docs 对应字段并排发生正确变化;
3. verifier 显示目标变更全部发生、无关对象零变化;
4. 外部应用出现并发修改后，任务面显式标红冲突并进入 reconciliation。
只展示“Agent 进度、当前步骤、暂停、日志”会把系统误读成 Sidekick/Magentic-UI；只展示“生成了漂亮表单”会被读成 DuetUI/Macaron；只展示“统一多个 App”会被读成 A-Mash/AOHP。

9 A2UI 协议版本：与撞车地图相互独立的工程决策

9.1 事实更新

A2UI 官方版本序列目前为：

v0.8 (Legacy) → v0.9 (Previous Stable) → v0.9.1 (Current) → v1.0 (Candidate).

官方 Evolution Guide 将 v0.9 描述为从“Structured Output First”到“Prompt First / In-Context Schema”的哲学级迁移：消息类型更名、组件结构扁平化、普通 JSON data model、可注入 catalog rules 与结构化 validation feedback。[15, 16]

9.2 对 TaskVM 的推荐

1. W2 起把 v0.9.1 设为正式 target。
2. 冻结 W1 的 v0.8 结果为 regression fixture，并保留薄适配器。
3. 不直接采用 v1.0 Candidate。

4. 使用 “v0.9.1 envelope + 冻结版本的 TaskVM Minimal Catalog”，不把官方 Basic Catalog 全集或 A2UI schema 变成内部 task-state 表示。
5. 在迁移 gate 中比较 schema-valid rate、repair 次数、token、UI 语义等价性、round-trip fidelity 与 non-interference。

关键边界

A2UI 是可替换的呈现协议，不是 TaskVM 的内部状态表示、执行语义或可信根。协议 ACK 也不能替代独立 verifier 对真实 Calendar/TaskBoard/Drive ground truth 的检查。

10 落地行动清单

10.1 Related Work 主文结构

1. **Generated Interaction during GUI-Agent Execution:** Morae、AgentLens、ALLOY。
2. **Persistent Generative and Malleable Task Interfaces:** Jelly、DuetUI、SaC、Macaron-A2UI、TaskLens。
3. **Cross-Application Composition:** AOHP、A-Mash、SUGILITE/PUMICE。
4. **Human-GUI-Agent Collaboration and Oversight:** Sidekick、Magentic-UI、CowPilot/MIWA。
5. **Protocols, Execution Backends, and Evaluation:** A2UI、MCP Apps/AG-UI、UFO2、OpenApps、MyPCBench 等。

10.2 W2 开工指令应增加的三项

1. A2UI 组件白名单和协议版本需基于官方 v0.9.1 原文重新核实；
2. 保留 v0.8 regression adapter，用同一批 W1 cases 做 compatibility gate；
3. related-work README 中加入 S/A 分级和每篇的 “不能 claim / 切割句”，避免 coding/demo 叙事回退。

10.3 下一轮文献审计优先级

1. 下载并逐页核对 Morae、AgentLens、ALLOY 的系统图、动作空间与 limitation；
2. 对 AOHP 的 Generated Service Entrances 做原文段落级对照；
3. 对 TaskLens、MAESTRO、Spatula 判断其 UI 是否真正运行时生成、是否与原软件状态双向绑定；
4. 对所有工程 demo 只记录 “公开可证的能力”，不要从产品宣传推导学术机制；
5. 在投稿前再做一次 2026-08 至截稿日的增量检索。

11 观察名单：保留但暂不作为撞车证据

以下名称来自前序广泛扫描。因公开材料不足、拓扑方向不同、或尚未完成 primary-source 核验，本报告不把它们用于 “prior work has already done X” 的强结论。

条目	当前暂定位置	需核验的问题
Generative UI Computer-Use Agent with Lang-Graph.js	工程 demo	动态 UI 是否为任务语义控件，还是 execution console?
AppLess / OpenUI demo	工程 GenUI	是否真正操作未修改第三方软件并持续绑定状态?
Moldable	生成式个人软件	受控平台内 API 是否可等同任意既有 GUI? 不可直接等同
OpenGenerativeUI	框架	是否有独立学术系统与真实 GUI write-back?
WebVIA	UI 复刻/开发	生成 UI 是否持续控制原网站?
WebGen-Agent	生成网站 + GUI 测试	人是否通过生成 UI 控制既有软件?
AUI-Gym	CUA 评测生成 UI	控制拓扑与 TaskVM 相反
InfiniteWeb	生成 GUI 环境	目标是训练 Agent，不是给人任务投影
ViMo	生成未来 GUI observation	UI 给 Agent 做 world model，不给人操作
MAIC-UI / LiveFigure / DrawAI	可编辑生成工件	生成的是内容/应用工件，不是多 App 状态控制面
GraphFlow / Ply	可执行工作流	程序表示，不是 live state projection
MAGUS 等 GenUI workshop 论文	早期议程	是否有真实 write-back、GT 和系统评测?
OS-Kairos / VeriOS / Mobile-Aptus	Agent 监督与恢复	是否存在 Agent-generated user-facing UI?
WindowsWorld	benchmark	具体跨应用覆盖、是否适合 app-substitution 实验
HiSA	trajectory state abstraction	抽象是否只供 Agent 推理，而非用户控制
Portal UX Agent	bounded component UI	是否有 external-app execution
Generative UI Accessibility Bridge	替代 UI	是否有双向写回和 GUI Agent
CHI 2026 GenUI workshop position papers	research agenda	只作为趋势，不作为完成系统

参考文献

[1] TaskVM Project Team. *TaskVM* 开工大纲 (单一权威文档 · 锁定版). Internal planning document, 2026.

- [2] Yi-Hao Peng, Dingzeyu Li, Jeffrey P. Bigham, and Amy Pavel. Morae: Proactively Pausing UI Agents for User Choices. In *Proceedings of UIST 2025*, 2025. DOI: [10.1145/3746059.3747797](https://doi.org/10.1145/3746059.3747797).
- [3] Jeonghyeon Kim, Byeongjun Joung, Junwon Lee, Joohyung Lee, Taehoon Min, and Sunjae Lee. AgentLens: Adaptive Visual Modalities for Human-Agent Interaction in Mobile GUI Agents. arXiv:2604.20279, 2026. <https://arxiv.org/abs/2604.20279>.
- [4] Jiawen Li, Zheng Ning, Yuan Tian, and Toby Jia-jun Li. ALLOY: Generating Reusable Agent Workflows from User Demonstration. arXiv:2510.10049, 2025. <https://arxiv.org/abs/2510.10049>.
- [5] Shanhui Zhao et al. AOHP: An Open-Source OS-Level Agent Harness for Personalized, Efficient and Secure Interaction. arXiv:2606.23449, 2026. <https://arxiv.org/abs/2606.23449>.
- [6] Sunjae Lee et al. A-Mash: Providing Single-App Illusion for Multi-App Use through User-centric UI Mashup. In *Proceedings of ACM MobiCom 2022*, pp. 690–702, 2022. DOI: [10.1145/3495243.3560522](https://doi.org/10.1145/3495243.3560522).
- [7] Yining Cao, Peiling Jiang, and Haijun Xia. Generative and Malleable User Interfaces with Generative and Evolving Task-Driven Data Model. In *Proceedings of CHI 2025*, Article 686, 2025. DOI: [10.1145/3706598.3713285](https://doi.org/10.1145/3706598.3713285).
- [8] Yuan Xu, Shaowen Xiang, Yizhi Song, Ruoting Sun, and Xin Tong. DuetUI: A Bidirectional Context Loop for Human-Agent Co-Generation of Task-Oriented Interfaces. arXiv:2509.13444, 2025. <https://arxiv.org/abs/2509.13444>.
- [9] Mulong Xie and Yang Xie. Software as Content: Dynamic Applications as the Human-Agent Interaction Layer. arXiv:2603.21334, 2026. <https://arxiv.org/abs/2603.21334>.
- [10] Fancy Kong et al. Macaron-A2UI: A Model for Generative UI in Personal Agents. arXiv:2605.24830, 2026. <https://arxiv.org/abs/2605.24830>.
- [11] Yimeng Liu and Misha Sra. TaskLens: Generating Task-Conditioned Scaffolded Interfaces for Learning Professional Creative Software. In *Proceedings of DIS 2026*, pp. 17–38, 2026. DOI: [10.1145/3800645.3813081](https://doi.org/10.1145/3800645.3813081).
- [12] Sangwook Lee, Sang Won Lee, Adnan Abbas, Young-Ho Kim, and Yan Chen. MAESTRO: Adapting GUIs and Guiding Navigation with User Preferences in Conversational Agents with GUIs. arXiv:2604.06134, 2026. <https://arxiv.org/abs/2604.06134>.
- [13] Boyu Li, Linjie Qiu, Lin-Ping Yuan, Duotun Wang, Yue Jiang, Zeyu Wang, and Hongbo Fu. Spatula: Exploring On-Demand In-Situ Interfaces and Interaction for Attribute Control. arXiv:2607.10405, 2026. <https://arxiv.org/abs/2607.10405>.
- [14] Yimeng Liu, Misha Sra, Jeevana Priya Inala, and Chenglong Wang. ReUseIt: Synthesizing Reusable AI Agent Workflows for Web Automation. arXiv:2510.14308, 2025. <https://arxiv.org/abs/2510.14308>.
- [15] A2UI Project. A2UI Protocol Evolution Guide: v0.8.1 to v0.9. <https://a2ui.org/specification/v0.9-evolution-guide/>, accessed 2026-08-06.
- [16] A2UI Project. A2UI Specification Versions. <https://a2ui.org/>, accessed 2026-08-06.